

Topic: Week 7 – SQL Query Implementation for E-Commerce Database

Scenario:

The e-commerce company stores customer details such as customer ID, name, email, and phone number. The management team uses SQL queries to retrieve, filter, sort, and count customer records for efficient customer management.

1. Display All Customer Details

```
SQL> SELECT * FROM Customer;
```

CUSTOMER_ID	CUSTOMER_NAME	EMAIL	PHONE
1001	Arun	arun@gmail.com	9876543210
1002	Priya	priya@gmail.com	9876543211
1003	Kumar	kumar@gmail.com	9876543212

CUSTOMER_ID	CUSTOMER_NAME	EMAIL	PHONE
1004	Divya	divya@gmail.com	9876543213
1005	Rahul	rahul@gmail.com	9876543214

2. Display all available products

```
SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P1001	Laptops	Cat0001	67500	2
P1002	Mobile	Cat0001	25000	10
P1003	TShirt	Cat0002	800	20
P1004	Shoes	Cat0003	1500	15

3.Retrieve product names and prices only:

```
SQL> SELECT Product_Name, Price FROM Product;
```

PRODUCT_NAME	PRICE
Laptops	67500
Mobile	25000
TShirt	800
Shoes	1500

4.Display All Orders Placed by Customers

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STAT	CUSTOMER_ID
1001	27-SEP-26	67500	Pending	1001
1002	27-SEP-26	25000	Shipped	1002
1003	27-SEP-26	1500	Delivered	1003
1004	27-SEP-26	4200	Pending	1004
1005	27-SEP-26	8000	Delivered	1005

5.Retrieve Payment Details

```
SQL> SELECT * FROM Payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST
TRANSACTION_AMOUNT				
1	1001	27-SEP-26	UPI	Success
	67500			
2	1002	27-SEP-26	Card	Success
	25000			
3	1003	28-SEP-26	Cash	Success
	1500			
TRANSACTION_AMOUNT				
4	1004	28-SEP-26	Net Banking	Pending
	4200			
5	1005	29-SEP-26	UPI	Success
	8000			
6	1001	29-SEP-26	Card	Failed
	1000			
TRANSACTION_AMOUNT				
7	1002	30-SEP-26	UPI	Pending
	5000			
8	1003	30-SEP-26	Net Banking	Failed
	2000			

8 rows selected.

2.Apply Filtering Conditions Using WHERE

1.Find Products with Price Greater Than ₹5000

```
SQL> SELECT * FROM Product WHERE Price > 5000;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P1001	Laptops	Cat0001	67500	2
P1002	Mobile	Cat0001	25000	10

2.Display Products Available in Stock

```
SQL> SELECT * FROM Inventory WHERE Stock_Status = 'Available';
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATUS	LAST_UPDA
I001	P1001	S001	20	Available	20-SEP-26
I002	P1002	S002	15	Available	20-SEP-26
I003	P1003	S003	10	Available	20-SEP-26

3.Display Completed Orders

```
SQL> SELECT * FROM Orders WHERE Order_Status = 'Delivered';
```

ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STAT	CUSTOMER_ID
1003	27-SEP-26	1500	Delivered	1003
1005	27-SEP-26	8000	Delivered	1005

Find Products with Rating Above 4

```
SQL> SELECT P.Product_Name, ROUND(AVG(R.Rating), 2) AS Average_Rating FROM Product P JOIN Review R ON P.Product_ID = R.Product_ID GROUP BY P.Product_ID, P.Product_Name HAVING AVG(R.Rating) > 4 ORDER BY Average_Rating DESC;
```

PRODUCT_NAME	AVERAGE_RATING
TShirt	5
Laptops	5

3.SORT DATA USING ORDER BY

1.Display Products from Lowest to Highest Price

```
SQL> SELECT * FROM Product ORDER BY Price ASC;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P1003	TShirt	Cat0002	800	20
P1004	Shoes	Cat0003	1500	15
P1002	Mobile	Cat0001	25000	10
P1001	Laptops	Cat0001	67500	2

2.Display Customers Alphabetically

```
SQL> SELECT * FROM Customer ORDER BY Customer_Name ASC;
```

CUSTOMER_ID	CUSTOMER_NAME
-------------	---------------

EMAIL

PHONE

1001 Arun
arun@gmail.com

9876543210

1004 Divya
divya@gmail.com

9876543213

1003 Kumar
kumar@gmail.com

9876543212

CUSTOMER_ID	CUSTOMER_NAME
-------------	---------------

EMAIL

PHONE

1002 Priya
priya@gmail.com

9876543211

1005 Rahul
rahul@gmail.com

9876543214

3.Find top expensive products

```
SQL> SELECT * FROM Product ORDER BY Price DESC;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
------------	--------------	---------	-------	----------------

P1001	Laptops	Cat0001	67500	2
-------	---------	---------	-------	---

P1002	Mobile	Cat0001	25000	10
-------	--------	---------	-------	----

P1004	Shoes	Cat0003	1500	15
-------	-------	---------	------	----

P1003	TShirt	Cat0002	800	20
-------	--------	---------	-----	----

4.Display Latest Orders First

```
SQL> SELECT * FROM Orders ORDER BY Order_Date DESC;
```

ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STAT	CUSTOMER_ID
----------	-----------	--------------	------------	-------------

1005	27-SEP-26	8000	Delivered	1005
------	-----------	------	-----------	------

1004	27-SEP-26	4200	Pending	1004
------	-----------	------	---------	------

1003	27-SEP-26	1500	Delivered	1003
------	-----------	------	-----------	------

1002	27-SEP-26	25000	Shipped	1002
------	-----------	-------	---------	------

1001	27-SEP-26	67500	Pending	1001
------	-----------	-------	---------	------

4.Retrieve unique values using DISTINCT

1.Display unique product categories

```
SQL> SELECT DISTINCT Category_ID FROM Product;

CATEGOR
-----
Cat0001
Cat0002
Cat0003
```

2.Find Different Payment Methods Used by Customers

```
SQL> SELECT DISTINCT Payment_Mode AS Payment_Method FROM Payment;

PAYMENT_METHOD
-----
UPI
Card
Cash
Net Banking
```

5.Search Products Based on Conditions

1.Find Products Within a Specific Price Range Using BETWEEN – Display products priced between ₹20,000 and ₹40,000.

```
SQL> SELECT * FROM Product WHERE Price BETWEEN 20000 AND 40000;

PRODUCT_ID  PRODUCT_NAME      CATEGOR      PRICE  STOCK_QUANTITY
-----
P1002       Mobile            Cat0001      25000           10
```

2.Display Products Belonging to a Particular Category

```
SQL> SELECT * FROM Product WHERE Category_ID = 'Cat0001';

PRODUCT_ID  PRODUCT_NAME      CATEGOR      PRICE  STOCK_QUANTITY
-----
P1001       Laptops           Cat0001      67500           2
P1002       Mobile            Cat0001      25000          10
```

3.Find Products Currently Available in Inventory

```
SQL> SELECT * FROM Inventory WHERE Stock_Quantity > 0;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATUS	LAST_UPDA
I001	P1001	S001	20	Available	20-SEP-26
I002	P1002	S002	15	Available	20-SEP-26
I003	P1003	S003	10	Available	20-SEP-26

4.Search Products Using Product Name

```
SQL> SELECT * FROM Product WHERE Product_Name = 'Mobile';
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P1002	Mobile	Cat0001	25000	10

5.Find Low-Stock Products

```
SQL> SELECT * FROM Product WHERE Stock_Quantity < 5;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P1001	Laptops	Cat0001	67500	2

6.Retrieve customer and product information

1.Display Customer Details with Their Orders

```
SQL> SELECT C.Customer_ID, C.Customer_Name, O.Order_ID, O.Order_Date, O.Total_Amount, O.Order_Status FROM Customer C JOIN Orders O ON C.Customer_ID = O.Customer_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID
1001	Arun	1001
27-SEP-26	67500 Pending	
1002	Priya	1002
27-SEP-26	25000 Shipped	
1003	Kumar	1003
27-SEP-26	1500 Delivered	

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID
1004	Divya	1004
27-SEP-26	4200 Pending	
1005	Rahul	1005
27-SEP-26	8000 Delivered	

2.Display product details with category information

```
SQL> SELECT P.Product_ID, P.Product_Name, C.Category_Name, P.Price, P.Stock_Quantity FROM Product P JOIN Category C ON P.Category_ID = C.Category_ID;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY_NAME	PRICE
STOCK_QUANTITY			
P1001	Laptops 2	Electronics	67500
P1002	Mobile 10	Electronics	25000
P1003	TShirt 20	Clothing	800
PRODUCT_ID PRODUCT_NAME CATEGORY_NAME PRICE			
STOCK_QUANTITY			
P1004	Shoes 15	Footwear	1500

3.Find customers who purchased a specific product.

```
SQL> SELECT DISTINCT C.Customer_ID, C.Customer_Name, O.Order_ID, OD.Product_ID FROM Customer C JOIN Orders O ON C.Customer_ID = O.Customer_ID JOIN Order_Details OD ON O.Order_ID = OD.Order_ID WHERE OD.Product_ID = 'P1001';
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID
PRODUCT		
	1001 Arun	1001
P1001		

4.Display products purchased by each customer

```
SQL> SELECT C.Customer_Name, O.Order_ID, P.Product_Name, OD.Quantity FROM Customer C JOIN Orders O ON C.Customer_ID = O.Customer_ID JOIN Order_Details OD ON O.Order_ID = OD.Order_ID JOIN Product P ON OD.Product_ID = P.Product_ID;
```

CUSTOMER_NAME		ORDER_ID
PRODUCT_NAME	QUANTITY	
Arun		1001
Laptops	1	
Priya		1002
Mobile	1	
Kumar		1003
TShirt	2	
CUSTOMER_NAME		ORDER_ID
PRODUCT_NAME	QUANTITY	
Divya		1004
Shoes	2	

7.Apply mutiple filtering conditions

1.Find electronics products costing more than rs 10000

```
SQL> SELECT P.Product_ID, P.Product_Name, P.Category_ID, P.Price FROM Product P WHERE P.Category_ID = 'Cat0001' AND P.Price > 10000 ORDER BY P.Price DESC;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE
P1001	Laptops	Cat0001	67500
P1002	Mobile	Cat0001	25000

2.Search Products Containing the Word “Mobile”

```
SQL> SELECT Product_ID, Product_Name, Price, Stock_Quantity FROM Product WHERE UPPER(Product_Name) LIKE '%MOBILE%';
```

PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK_QUANTITY
P1002	Mobile	25000	10

3.Find Orders Placed Within a Specific Date Range

```
SQL> SELECT * FROM Orders WHERE Order_Date >= DATE '2026-09-27' AND Order_Date < DATE '2026-09-28' ORDER BY Order_Date;
```

ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STAT	CUSTOMER_ID
1001	27-SEP-26	67500	Pending	1001
1002	27-SEP-26	25000	Shipped	1002
1003	27-SEP-26	1500	Delivered	1003
1004	27-SEP-26	4200	Pending	1004
1005	27-SEP-26	8000	Delivered	1005

8.Generate basic business reports

Report 1: product availability report

Display :

1.product name

2.price

3.stock quantity

4.availability status

```
SQL> SELECT Product_Name, Price, Stock_Quantity, CASE WHEN Stock_Quantity > 0 THEN 'Available' ELSE 'Out of Stock' END AS Availability_Status FROM Product;
```

PRODUCT_NAME	PRICE	STOCK_QUANTITY	AVAILABILITY
Laptops	67500	2	Available
Mobile	25000	10	Available
TShirt	800	20	Available
Shoes	1500	15	Available

Report 2: customer report

Total number of customers

```
SQL> SELECT COUNT(*) AS Total_Customers FROM Customer;
```

TOTAL_CUSTOMERS
5

New customer registrations

```
SQL> SELECT Customer_ID, Customer_Name, Email FROM Customer ORDER BY Customer_ID DESC;
```

CUSTOMER_ID	CUSTOMER_NAME	EMAIL
1005	Rahul	rahul@gmail.com
1004	Divya	divya@gmail.com
1003	Kumar	kumar@gmail.com

CUSTOMER_ID	CUSTOMER_NAME	EMAIL
1002	Priya	priya@gmail.com
1001	Arun	arun@gmail.com

Report 3: order report

display:

Total orders

```
SQL> SELECT COUNT(*) AS Total_Orders FROM Orders;
```

TOTAL_ORDERS
5

Completed orders

```
SQL> SELECT COUNT(*) AS Completed_Orders FROM Orders WHERE Order_Status = 'Delivered';
```

COMPLETED_ORDERS
2

Pending orders

```
SQL> SELECT COUNT(*) AS Pending_Orders FROM Orders WHERE Order_Status = 'Pending';
```

PENDING_ORDERS
2

Cancelled orders

```
SQL> SELECT COUNT(*) AS Cancelled_Orders FROM Orders WHERE Order_Status = 'Cancelled';
```

CANCELLED_ORDERS
0

Report 4: product performance report

Highest priced products

```
SQL> SELECT Product_ID, Product_Name, Price FROM Product ORDER BY Price DESC;
```

PRODUCT_ID	PRODUCT_NAME	PRICE
P1001	Laptops	67500
P1002	Mobile	25000
P1004	Shoes	1500
P1003	TShirt	800

Most reviewed products

```
SQL> SELECT P.Product_ID, P.Product_Name, COUNT(R.Product_ID) AS Total_Reviews FROM Product P JOIN Review R ON P.Product_ID = R.Product_ID  
GROUP BY P.Product_ID, P.Product_Name ORDER BY Total_Reviews DESC;
```

PRODUCT_ID	PRODUCT_NAME	TOTAL_REVIEWS
P1001	Laptops	2
P1003	TShirt	1
P1004	Shoes	1
P1002	Mobile	1

Available products

```
SQL> SELECT Product_ID, Product_Name, Stock_Quantity FROM Product WHERE Stock_Quantity > 0;
```

PRODUCT_ID	PRODUCT_NAME	STOCK_QUANTITY
P1001	Laptops	2
P1002	Mobile	10
P1003	TShirt	20
P1004	Shoes	15